

УТВЕРЖДАЮ

Руководитель организации-заказчика

DV Group

_____ И.О. Фамилия

«___» _____ 2026 г.

**АВТОМАТИЗИРОВАННАЯ СИСТЕМА УПРАВЛЕНИЯ ПРОДАЖАМИ
ОБЪЕКТОВ НЕДВИЖИМОСТИ
«CRM Недвижимость»**

РУКОВОДСТВО ПРОГРАММИСТА

ЛИСТ УТВЕРЖДЕНИЯ

UZ.XXXXXXXXXX.00001-01 33 01-ЛУ

СОГЛАСОВАНО

Руководитель организации-разработчика

Организация-разработчик

_____ И.О. Фамилия

«___» _____ 2026 г.

РАЗРАБОТЧИКИ

Ответственный исполнитель

Организация-разработчик

_____ И.О. Фамилия

«___» _____ 2026 г.

Ташкент 2026

УТВЕРЖДЕН
UZ.XXXXXXXXXX.00001-01 33 01-ЛУ

**АВТОМАТИЗИРОВАННАЯ СИСТЕМА УПРАВЛЕНИЯ ПРОДАЖАМИ
ОБЪЕКТОВ НЕДВИЖИМОСТИ
«CRM Недвижимость»**

РУКОВОДСТВО ПРОГРАММИСТА

UZ.XXXXXXXXXX.00001-01 33 01

Листов 21

Ташкент 2026

АННОТАЦИЯ

Документ предназначен для программистов, сопровождающих и развивающих АС «CRM Недвижимость». Приведены назначение и характеристики программы, структура исходных текстов, правила построения серверной и клиентской частей (разграничение доступа, изоляция компаний, мягкое удаление, журналы, транзакции, выдача файлов, локализация), порядок обращения к программному интерфейсу и публичному API, формат входных и выходных данных, сообщения программы, а также порядок внесения и проверки изменений.

Перечень маршрутов программного интерфейса приведён в приложении А.

СОДЕРЖАНИЕ

ПЕРЕЧЕНЬ ТЕРМИНОВ И СОКРАЩЕНИЙ.....	5
1 Назначение и условия применения программы.....	7
1.1 Назначение	7
1.2 Условия применения	7
2 Характеристики программы	8
2.1 Структура исходных текстов.....	8
2.2 Разграничение доступа.....	8
2.2.1 Добавление нового ресурса	9
2.3 Изоляция компаний	9
2.4 Мягкое удаление	10
2.5 Журналы изменений.....	10
2.6 Транзакции и конкурентный доступ.....	10
2.7 Выдача файлов	10
2.8 Постраничная выдача и ошибки	11
2.9 Кэш и Redis.....	11
2.10 Локализация клиентской части	11
2.11 Валюты и курсы	12
2.12 Регламентные задания	12
3 Обращение к программе	13
3.1 Программный интерфейс для клиентского приложения	13
3.2 Публичный API для партнёров.....	13
4 Входные и выходные данные	14
5 Сообщения.....	15
5.1 Коды ответов	15
5.2 Порядок внесения изменений.....	15
Приложение А (справочное) Маршруты программного интерфейса.....	16
А.1 Перечень маршрутов	16

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ	21
----------------------------------	----

ПЕРЕЧЕНЬ ТЕРМИНОВ И СОКРАЩЕНИЙ

Таблица 1 – Сокращения

Сокращение	Расшифровка
АС	Автоматизированная система
БД	База данных
ВК	Внешний ключ таблицы базы данных
ЕСПД	Единая система программной документации
ЖК	Жилой комплекс
ИО	Информационное обеспечение
ОС	Операционная система
ПИНФЛ	Персональный идентификационный номер физического лица
ПК	Первичный ключ таблицы базы данных
ПО	Программное обеспечение
РОП	Руководитель отдела продаж
СУБД	Система управления базами данных
ТЗ	Техническое задание
УТП	Уникальное торговое предложение
API	Программный интерфейс приложения (Application Programming Interface)
CRM	Система управления взаимоотношениями с клиентами (Customer Relationship Management)
HTTPS	Протокол передачи гипертекста с шифрованием
JWT	Формат токена доступа (JSON Web Token)
REST	Архитектурный стиль программного интерфейса поверх HTTP
UZS, USD, EUR	Коды валют: узбекский сум, доллар США, евро

Таблица 2 – Термины

Термин	Определение
Бронь	Сделка в начальном статусе: объект закреплён за клиентом до даты окончания брони и выведен из продажи
График платежей	Разбивка стоимости по договору на платежи с плановыми датами; сумма платежей равна стоимости по договору
Заявка	Зарегистрированное обращение клиента с его интересом к приобретению недвижимости; основная единица воронки продаж
Клиент	Физическое лицо, с которым работает компания; карточка клиента принадлежит одной компании
Компания	Юридическое лицо, работающее в системе; данные и справочники разных компаний изолированы друг от друга
Мягкое удаление	Пометка записи удалённой без физического удаления: запись скрывается из списков и отчётов и может быть восстановлена
Область видимости	Граница данных, доступных пользователю по разрешению: свои записи, отдел, компания или вся система
Объект недвижимости, объект	Квартира, коммерческое помещение, парковочное место, кладовая или коттедж в составе дома
Партнёр	Внешняя система (сайт, бот, агентство), обращающаяся к публичному API по API-ключу
Разрешение	Сочетание действия, ресурса и области видимости, например «просмотр клиентов своего отдела»
Расторжение	Прекращение сделки, по которой прошли платежи или есть подпись клиента; требует документа-основания и влечёт возврат денег
Роль	Именованный набор разрешений, назначаемый пользователям
Сделка	Договорные отношения с клиентом по одному объекту: от брони до подписанного договора или отмены
Справочник	Перечень значений для выбора в формах (статусы заявок, причины отказа, типы платежей и т.п.)
Шахматка	Табличное представление объектов дома: подъезды по столбцам, этажи по строкам, статус — цветом ячейки

1 Назначение и условия применения программы

1.1 Назначение

Программа автоматизирует продажи объектов недвижимости. Перечень функций — в документе «Описание автоматизированных функций»; коды функций (ОРГ-01, СДЕ-04 и т.д.) рекомендуется указывать в задачах и сообщениях коммитов.

1.2 Условия применения

Среда исполнения — по документу «Руководство системного программиста». Среда разработки:

- Python 3.12 и виртуальное окружение в каталоге `backend/.venv` (`pip install -r requirements.txt`);
- Node.js 20 и зависимости клиентской части (`npm ci` в каталоге `frontend-new`);
- для локального запуска серверной части без Docker — переменная `USE_SQLITE=True` (БД SQLite) и локальный Redis либо замена кэша (см. 2.9);
- для запуска комплекса в режиме разработки — `docker compose -f docker-compose.dev.yml up` (горячая перезагрузка клиентской части через `nginx` в режиме `DEV_MODE`).

2 Характеристики программы

2.1 Структура исходных текстов

Таблица 3 – Каталог репозитория

Каталог	Содержимое
backend/real_estate_project	Настройки, корневые маршруты, выдача файлов, страничная выдача
backend/permissions	Структура, пользователи, права, аутентификация, API-ключи
backend/apps/{realty,crm,deals,finances,documents,tasks,reports}	Прикладные подсистемы: models.py, serializers.py, views.py, urls.py, filters.py, migrations, management/commands, tests.py
frontend-new/src	Клиентская часть (документ «Описание программного обеспечения», таблица 5)
nginx	Конфигурация обратного прокси
docs	Ролевые инструкции (docs/_src, сборка docs/_src/build.sh) и проектная документация (docs/gost)

2.2 Разграничение доступа

Разрешение — запись `permissions.Permission` с полями `action`, `resource`, `scope`. Роль (`permissions.Role`) — набор разрешений; профиль пользователя (`permissions.UserProfile`) — набор ролей, компания и отдел. Проверки выполняются функциями модуля `permissions/backends.py`:

– `can_user_perform_action(user, action, resource_type, obj=None, scope=None)` — может ли пользователь выполнить действие; с `obj` дополнительно проверяется, входит ли объект в область разрешения;

– `get_filtered_queryset(user, queryset, resource_type, max_scope=None)` — отбор записей по наибольшей области видимости пользователя для ресурса;

– `get_user_max_scope(user, resource_type)` — наибольшая область пользователя.

На уровне программного интерфейса применяются классы `permissions/permissions.py`, наследники `BaseResourcePermission` с атрибутом `resource_type`: метод HTTP отображается в действие (GET — VIEW, POST — ADD, PUT и PATCH — EDIT, DELETE — DELETE).

Правила:

а) каждое представление, выдающее записи, обязано отбирать их через `get_filtered_queryset`; проверка только класса разрешения не ограничивает данные областью видимости;

б) суперпользователь и профиль с признаком `is_system_admin` проходят все проверки;

в) интерфейс скрывает недоступное (`frontend-new/src/utils/permissions.ts: hasPermission, isAdmin, getMaxScope`), но сервер проверяет всё самостоятельно.

2.2.1 Добавление нового ресурса

а) Добавить код в `Permission.Resource (permissions/models.py)` и создать миграцию.

б) Выполнить `python manage.py init_permissions` — будут созданы разрешения по всем действиям и областям; для включения в типовые роли изменить `init_permissions.py` и выполнить с `--sync-roles`.

в) Создать класс разрешения — наследник `BaseResourcePermission` с `resource_type`.

г) В представлениях использовать этот класс и `get_filtered_queryset`.

д) В клиентской части добавить код в тип `ResourceType (utils/permissions.ts)`, перевод `resources.<КОД>` во все три файла локализации и, при необходимости, пункт меню (`layouts/RootLayout.tsx`) или вкладку настроек (`utils/settingsTabs.ts`).

2.3 Изоляция компаний

а) Каждая бизнес-модель и модель справочника имеет поле `company`.

б) При создании записи компания берётся из профиля пользователя (`request.user.profile.company`), а не из входных данных.

в) Ссылки на связанные записи во входных данных (клиент, объект, счёт получателя, тип платежа, ответственный) проверяются в сериализаторе на принадлежность компании пользователя; ссылка на запись другой компании — ошибка проверки.

г) Уникальность значений, видимых пользователю (адрес электронной почты клиента, номер договора, название записи справочника), задаётся в пределах компании (`unique_together с company`).

д) Справочники подключают `CompanyScopedReferenceMixin (permissions/reference_scope.py)`: выдаются записи своей компании и общесистемные (`company IS NULL`); общесистемные создаёт и изменяет только администратор.

е) Записи, создаваемые системой автоматически (журналы, встречи при регистрации клиента, клиенты из публичного API), также получают компанию.

Команда `check_company_isolation` показывает записи без компании.

2.4 Мягкое удаление

Модели заявок и встреч имеют поля `is_deleted`, `deleted_at`, `deleted_by` и методы `soft_delete(user)` и `restore()`. Представления подключают `SoftDeleteMixin` (`apps/crm/views.py`): `perform_destroy` помечает запись и пишет журнал, `filter_deleted` исключает удалённые записи, если не передан параметр `include_deleted=true`. Восстановление — маршруты `.../<id>/restore/`. Все выборки для отчётов и сводных таблиц обязаны исключать удалённые записи.

2.5 Журналы изменений

Для журналируемых моделей существует модель журнала (`ApplicationLog`, `ClientLog`, `MeetingLog`, `DealLog`, `PaymentLog`, `BuildingLog`, `DiscountLog`, `TaskLog`) с полями записи, пользователя, времени и текста действия. Запись журнала создаётся в представлении при каждом значимом изменении; для полей с изменёнными значениями используется формулировка «Поле `'...'` изменено с `'...'` на `'...'`», которую разбирает компонент клиентской части `HumanizedLog`. Изменение этой формулировки требует согласованного изменения компонента.

2.6 Транзакции и конкурентный доступ

Изменения нескольких записей выполняются в `transaction.atomic()`. Создание брони блокирует запись объекта (`select_for_update`) и повторно проверяет её статус и наличие блокирующих сделок внутри транзакции — это исключает двойную бронь. Этот же приём применяется при любых изменениях статуса объекта.

2.7 Выдача файлов

Поля файлов сериализуются функцией `build_media_url(file_field)` (`real_estate_project/media_access.py`): для общедоступных каталогов (`projects`, `buildings`, `layouts`, `avatars`) возвращается прямая ссылка, для остальных — ссылка с подписью и сроком действия (`sign_media_path`). Маршрут `/media/<путь>` (`serve_media`) проверяет подпись и при `MEDIA_USE_ACCEL_REDIRECT=True` возвращает заголовок `X-Accel-Redirect` на `/protected-media/<путь>`, иначе отдаёт файл сам. Новый каталог файлов должен быть подключён в сервисы `backend` и `nginx` файла `docker-compose.yml`.

2.8 Постраничная выдача и ошибки

Списки выдаются целиком, если в запросе нет параметра `page`; при его наличии — постранично (`page_size` по умолчанию 50, не более 500; класс `OptionalPageNumberPagination`). Ответ при постраничной выдаче содержит `count`, `next`, `previous`, `results`.

Отказ в операции возвращается с кодом 400 (403, 404) и телом вида `{"error": "текст"}` либо `{"<поле>": ["текст"]}`. Текст должен объяснять пользователю причину и способ исправления. Клиентская часть извлекает его функцией `extractApiError` (`utils/apiError.ts`); новые формы должны использовать её, а не общее сообщение.

2.9 Кэш и Redis

Кэш Django (CACHES) размещён в Redis: в нём хранятся счётчики ограничения частоты запросов, неудачных попыток входа и отметки последнего пересчёта просрочки. В тестах кэш заменяется на `LocMemCache` декоратором `override_settings` — без этого тесты обращаются к Redis.

2.10 Локализация клиентской части

а) Загружаются только файлы `src/i18n/locales/ru.json`, `en.json`, `uz.json`. Каталоги `locales/{ru,en,uz}/*.json` — устаревший результат сценария `scripts/split-locales.cjs`, в приложение не подключаются; ключи с префиксами вида `permissions.*`, `buildings.*`, рассчитанные на них, не работают.

б) Любая надпись интерфейса выводится через `t('ключ')`; ключ должен существовать во всех трёх файлах. Второй аргумент `t('ключ', 'текст')` не использовать: он скрывает отсутствие перевода.

в) Коды классификаторов переводятся функциями `utils/translations.ts` (`translateDealStatus`, `translateResource` и др.); новое значение перечисления сервера требует перевода в ветках `statuses.*` или `resources.*` всех языков.

г) Таблицы MUI `DataGrid` выводятся через `components/common/LocalizedDataGrid.tsx`: для русского языка используется официальная локаль `ruRU`, для узбекского — собственный набор с именами ключей версии 8.

д) Сообщения сервера и тексты журналов формируются на русском языке.

2.11 Валюты и курсы

Пересчёт сумм между валютами выполняется только функциями модуля `apps/finances/currency.py`: `cross_rate` (кросс-курс на дату через сум), `convert` (сумма на дату), класс `Converter` (приведение множества сумм к одной валюте без обращения к ЦБ, с перечнем валют без курса — для отчётов и дашборда). Кросс-курс в вычислениях не округляется: у пары сум → доллар он порядка 0,00008, и округление до 8 знаков давало ошибку до нескольких долларов на договорах в миллиарды сумов. Для записи курса в сделку или платёж используется `stored_rate` (12 знаков). Итоговая сумма округляется функцией `money` до 0,01 по правилу «половина вверх».

Курс хранится в модели `ExchangeRate` как число сумов за единицу валюты; запись без компании — курс ЦБ, с компанией — ручной курс. Валюта сделок и поддерживаемые валюты компании возвращаются функциями `company_base_currency` и `supported_currencies`. Отсутствие курса сообщается исключением `RateUnavailable`, текст которого показывается пользователю.

2.12 Регламентные задания

Задания оформлены командами управления (`management/commands`) и запускаются контейнером `scheduler` раз в час. Новое задание добавляется командой управления и строкой в разделе `command` сервиса `scheduler`. Задание должно быть идемпотентным: повторный запуск не меняет уже обработанные записи.

3 Обращение к программе

3.1 Программный интерфейс для клиентского приложения

Все маршруты имеют префикс `/api/`. Аутентификация:

- а) `POST /api/permissions/auth/login/` с телом `{"username": "...", "password": "..."} —` ответ содержит токен доступа `access` и сведения о пользователе; токен обновления устанавливается в cookie `crm_refresh` (путь `/api/`, `HttpOnly`);
- б) последующие запросы — с заголовком `Authorization: Bearer <access>`;
- в) `POST /api/token/refresh/` (cookie передаётся браузером автоматически) — новый токен доступа и новая cookie;
- г) `POST /api/permissions/auth/logout/` — отзыв токена обновления и удаление cookie.

Полное описание маршрутов, параметров и схем ответов публикуется системой в формате OpenAPI 3: `/api/schema/swagger/`. Перечень маршрутов приведён в приложении А.

3.2 Публичный API для партнёров

Запросы выполняются по HTTPS с заголовком `X-API-Key: <ключ>`.

```
curl -H "X-API-Key: <ключ>" https://crm.example.uz/api/public/projects/

curl -X POST -H "X-API-Key: <ключ>" -H "Content-Type: application/json" \
  -d '{"full_name": "Иванов Иван", "phone_number": "+998901234567",
    "source": "INTERNET", "notes": "Интересует 2-комнатная"}' \
  https://crm.example.uz/api/public/applications/
```

Таблица 4 – Маршруты публичного API

Маршрут	Метод	Разрешённое действие ключа
<code>/api/public/projects/</code>	GET	VIEW PROJECTS
<code>/api/public/projects/<id>/</code>	GET	VIEW PROJECTS
<code>/api/public/buildings/<id>/</code>	GET	VIEW BUILDINGS
<code>/api/public/buildings/<id>/layouts/</code>	GET	VIEW LAYOUTS
<code>/api/public/applications/</code>	POST	CREATE APPLICATION

Значения поля `source`: `INTERNET`, `SOCIAL_MEDIA`, `OFFICE`, `CALL`. Поле `phone_number` обязательно; номер приводится к единому виду.

4 Входные и выходные данные

Входные и выходные данные программного интерфейса — объекты JSON в кодировке UTF-8; файлы передаются в запросах `multipart/form-data`. Даты — в формате ISO 8601 (ГГГГ-ММ-ДД, дата и время — с часовым поясом). Денежные суммы — десятичные числа в виде строки или числа с двумя знаками после запятой; валюта — отдельным полем, буквенным кодом из перечня системы (UZS, USD, EUR, RUB, CNY, KZT, GBP, AED, TRY). Цена объекта в публичном API сопровождается полем `price_currency` — валютой прайса проекта. Коды классификаторов — строки латиницей (документ «Описание информационного обеспечения системы», приложение Б).

Файловые входные и выходные данные: шаблоны XLSX объектов и планов (выдаются системой и принимаются обратно), шаблоны и сформированные документы DOCX, загрузки XLSX.

5 Сообщения

5.1 Коды ответов

Таблица 5 – Коды ответов программного интерфейса

Код	Значение
200, 201, 204	Успешное выполнение
400	Ошибка входных данных или нарушение бизнес-правила; тело содержит причину
401	Нет или истёк токен доступа (клиентская часть обновляет его автоматически); для публичного API — нет, неверный, отключённый или истёкший ключ
403	Нет разрешения, запись вне области видимости, вход заблокирован; для публичного API — действие или IP-адрес не разрешены ключу, запрос не по HTTPS
404	Запись не найдена или принадлежит другой компании
413	Превышен размер загружаемого файла
429	Превышено ограничение частоты запросов
500, 502	Внутренняя ошибка или недоступность серверной части

5.2 Порядок внесения изменений

а) Создать ветвь от основной ветви репозитория.

б) Внести изменения, соблюдая правила раздела 2; для изменений структуры данных — создать миграцию (`python manage.py makemigrations`), для данных — миграцию с RunPython.

в) Добавить или обновить автоматические тесты (`tests.py` приложения).

г) Выполнить проверки:

```
cd backend
python manage.py check
python manage.py makemigrations --check --dry-run
python manage.py test apps permissions --top-level-directory .
cd ../frontend-new
npx tsc -p tsconfig.app.json --noEmit
npm run build
```

а) Если изменилось поведение — обновить ролевые инструкции (`docs/_src/body-*.html`, затем `bash docs/_src/build.sh`) и проектную документацию (`docs/gost/src/*.md`; при изменении моделей или маршрутов — снимок `python ../docs/gost/tools/snapshot.py` из каталога `backend`; затем `python docs/gost/tools/build.py` и `tools/finalize.ps1`).

б) Передать изменения на проверку и испытания по документу «Программа и методика испытаний».

Примечание – Команда `manage.py test` без указания `apps permissions` пытается импортировать сценарии `test_*.py` из корня каталога `backend`, которые не являются модулями тестов, и сообщает об ошибках их загрузки. Указание меток приложений исключает эти сценарии.

Приложение А (справочное)

Маршруты программного интерфейса

А.1 Перечень маршрутов

Перечень сформирован автоматически по маршрутам серверной части (снимок docs/gost/data/routes.json). Обозначения <int:pk>, <id> — идентификатор записи.

Таблица А.1 – Маршруты API: служебные и аутентификация

Маршрут	Методы	Назначение
/api/health/	GET	Публичный эндпоинт для проверки состояния сервиса (Docker healthcheck)
/api/permissions/auth/login/	POST	Вход пользователя
/api/permissions/auth/logout/	POST	Выход пользователя
/api/permissions/auth/password-reset/	POST	Запрос на восстановление пароля
/api/permissions/auth/password-reset/confirm/	POST	Подтверждение сброса пароля и установка нового
/api/permissions/me/	GET, PATCH	Получение профиля текущего пользователя
/api/token/	POST	
/api/token/refresh/	POST	Обновление access-токена по refresh из httpOnly cookie
/media/<path:path>	GET	Отдаёт файл из MEDIA_ROOT с проверкой доступа
/api/schema/	GET	OpenApi3 schema for this API. Format can be selected via content negotiation
/api/schema/swagger/	GET	
/api/schema/redoc/	GET	

Таблица А.2 – Маршруты API: публичный api партнёров

Маршрут	Методы	Назначение
/api/public/projects/	GET	Публичный список проектов. Требуется валидный API-ключ партнёра
/api/public/projects/<int:pk>/	GET	Публичная детальная страница проекта. Требуется валидный API-ключ партнёра
/api/public/buildings/<int:pk>/	GET	Публичная детальная страница дома. Требуется валидный API-ключ партнёра
/api/public/buildings/<int:building_pk>/layouts/	GET	Публичный список планировок здания. Требуется валидный API-ключ партнёра
/api/public/applications/	POST	Публичный эндпоинт для создания заявок с сайтов партнёров

Таблица А.3 – Маршруты API: клиенты, заявки, встречи

Маршрут	Методы	Назначение
/api/dashboard/analytics/	GET	
/api/clients/	GET, POST	
/api/clients/<int:pk>/	DELETE, GET, PATCH, PUT	
/api/clients/<int:pk>/files/	GET, POST	
/api/applications/	GET, POST	
/api/applications/<int:pk>/	DELETE, GET, PATCH, PUT	
/api/applications/<int:pk>/restore/	POST	Возвращает удалённую заявку в работу
/api/applications/summary/	GET	
/api/users/	GET	
/api/application-statuses/	GET, POST	GET: Список всех статусов заявок (доступно всем авторизованным пользователям)
/api/application-statuses/<int:pk>/	DELETE, GET, PATCH, PUT	GET: Детали статуса заявки (доступно всем авторизованным пользователям)
/api/meetings/	GET, POST	
/api/meetings/<int:pk>/	DELETE, GET, PATCH, PUT	
/api/meetings/<int:pk>/restore/	POST	Возвращает удалённую встречу в работу
/api/meetings/summary/	GET	
/api/rejection-reasons/	GET, POST	

Таблица А.4 – Маршруты API: каталог недвижимости и скидки

Маршрут	Методы	Назначение
/api/projects/	GET, POST	
/api/projects/<int:pk>/	DELETE, GET, PATCH, PUT	
/api/buildings-all/	GET	Возвращает плоский список всех домов для использования в выпадающих списках
/api/projects/<int:project_pk>/buildings/	GET, POST	
/api/projects/<int:project_pk>/buildings/<int:pk>/	DELETE, GET, PATCH, PUT	
/api/projects/<int:project_pk>/buildings/<int:building_pk>/layouts/	GET, POST	
/api/projects/<int:project_pk>/buildings/<int:building_pk>/layouts/<int:pk>/	DELETE, GET, PATCH, PUT	
/api/projects/<int:project_pk>/buildings/<int:building_pk>/layouts/bulk-upload/	POST	Массовая загрузка изображений планировок
/api/building-types/	GET, POST	
/api/building-types/<int:pk>/	DELETE	
/api/projects/<int>/buildings/<int:building_pk>/properties/<int:pk>/	GET, PATCH, PUT	
/api/projects/<int:project_pk>/buildings/<int:building_pk>/download-template/	GET	
/api/discounts/	GET, POST	
/api/discounts/<int:pk>/	DELETE, GET, PATCH, PUT	
/api/projects/<int:project_pk>/buildings/<int:building_pk>/gallery/	POST	View для загрузки нового изображения в галерею дома
/api/projects/<int:project_pk>/buildings/<int:building_pk>/gallery/<int:pk>/	DELETE	View для удаления изображения из галереи дома
/api/projects/<int:project_pk>/buildings/<int:building_pk>/upload-properties/	POST	

Маршрут	Методы	Назначение
/api/projects/<int:project_pk>/gallery/	POST	View для загрузки нового изображения в галерею проекта
/api/projects/<int:project_pk>/gallery/<int:pk>/	DELETE	View для удаления изображения из галереи

Таблица A.5 – Маршруты API: сделки, платежи, документы

Маршрут	Методы	Назначение
/api/deals/	GET, POST	
/api/deals/<int:pk>/	GET, PATCH, PUT	
/api/deals/<int:deal_pk>/cancel/	POST	View для отмены или расторжения сделки
/api/deals/summary/	GET	
/api/deals/<int:deal_pk>/available-discounts/	GET	Возвращает список скидок, доступных для объекта в сделке
/api/finances/payments/	GET	
/api/finances/payment-types/	GET, POST	
/api/finances/payment-types/<int:pk>/	DELETE	
/api/finances/beneficiary-accounts/	GET, POST	
/api/finances/beneficiary-accounts/<int:pk>/	DELETE	
/api/finances/payments/<int:pk>/	GET, PATCH, PUT	View для обновления данных по конкретному платежу
/api/finances/payments/<int:pk>/mark-as-returned/	POST	
/api/deals/<int:deal_pk>/payment-schedule/	POST	Создаёт или обновляет график платежей сделки
/api/finances/summary/	GET	
/api/finances/currency-settings/	GET, PATCH	Валюта сделок и поддерживаемые валюты компании
/api/finances/exchange-rates/	GET, POST	GET — курсы на дату (по умолчанию сегодня) для поддерживаемых валют
/api/finances/exchange-rates/refresh/	POST	Загрузка актуальных курсов ЦБ по кнопке
/api/finances/exchange-rates/<int:pk>/	DELETE	Удаление ручного курса своей компании. Курсы ЦБ не удаляются
/api/templates/	GET, POST	
/api/templates/<int:pk>/	DELETE, GET, PATCH, PUT	
/api/deals/<int:deal_pk>/available-templates/	GET	
/api/deals/<int:deal_pk>/generate-document/<int:template_pk>/	GET	

Таблица A.6 – Маршруты API: отчёты и планы

Маршрут	Методы	Назначение
/api/reports/plan-fact/	GET	
/api/reports/plan-template/	GET	
/api/reports/plan-upload/	POST	
/api/reports/employee-plan-fact/	GET	
/api/reports/employee-plan-template/	GET	
/api/reports/employee-plan-upload/	POST	

Таблица А.7 – Маршруты API: задачи

Маршрут	Методы	Назначение
/api/tasks/	GET, POST	
/api/tasks/calendar/	GET	
/api/tasks/created_by_me/	GET	
/api/tasks/kanban/	GET	
/api/tasks/my_tasks/	GET	
/api/tasks/overdue/	GET	
/api/tasks/stats/	GET	
/api/tasks/<id>/	DELETE, GET, PATCH, PUT	
/api/tasks/<id>/add_comment/	POST	
/api/tasks/<id>/cancel/	POST	
/api/tasks/<id>/comments/	GET	
/api/tasks/<id>/complete/	POST	
/api/tasks/<id>/logs/<log_id>/	DELETE	
/api/tasks/<id>/logs/	GET	
/api/tasks/<id>/reopen/	POST	
/api/tasks/<id>/start/	POST	
/api/tasks/<id>/subtasks/	GET	
/api/task-comments/	GET, POST	
/api/task-comments/<id>/	DELETE, GET, PATCH, PUT	

Таблица А.8 – Маршруты API: организационная структура и права

Маршрут	Методы	Назначение
/api/permissions/companies/	GET, POST	
/api/permissions/companies/accessible/	GET	
/api/permissions/companies/<id>/	DELETE, GET, PATCH, PUT	
/api/permissions/departments/	GET, POST	
/api/permissions/departments/accessible/	GET	
/api/permissions/departments/<id>/	DELETE, GET, PATCH, PUT	
/api/permissions/permissions/	GET	
/api/permissions/permissions/grouped_by_resource/	GET	
/api/permissions/permissions/<id>/	GET	
/api/permissions/roles/	GET, POST	
/api/permissions/roles/<id>/	DELETE, GET, PATCH, PUT	
/api/permissions/roles/<id>/assign_permissions/	POST	
/api/permissions/roles/<id>/users/	GET	
/api/permissions/user-profiles/	GET, POST	
/api/permissions/user-profiles/create_user/	POST	
/api/permissions/user-profiles/<id>/	DELETE, GET, PATCH, PUT	
/api/permissions/user-profiles/<id>/ban/	POST	
/api/permissions/user-profiles/<id>/change_password/	POST	
/api/permissions/user-profiles/<id>/check_permission/	POST	
/api/permissions/user-profiles/<id>/permissions/	GET	
/api/permissions/user-profiles/<id>/soft_delete/	POST	
/api/permissions/user-profiles/<id>/unban/	POST	
/api/permissions/logs/	GET	
/api/permissions/logs/<id>/	GET	

Маршрут	Методы	Назначение
/api/permissions/partner-api-keys/	GET, POST	
/api/permissions/partner-api-keys/<id>/	DELETE, GET, PATCH, PUT	
/api/permissions/partner-api-keys/<id>/regenerate/	POST	
/api/permissions/partner-api-keys/<id>/toggle_active/	POST	
/api/permissions/stats/	GET	Статистика по разрешениям и ролям

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]